

Министерство науки и образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Логика и основы
алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнил:

студент группы 25BBB1
Цеплов Е.В.
Разживалов Н.А.

Приняли:

к.т.н., доцент Юрова О.В.,
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Оценка времени выполнения программ

Цель работы

Оценка времени выполнения программ

Лабораторное задание

Дана программа, вычисляющая произведение двух матриц.

Задание 1:

1. Вычислить порядок сложности программы (O -символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Даны реализации алгоритмов сортировки Шелла и быстрой сортировки.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Ход работы

Задание 1:

1. В программе используется три вложенных цикла

```

for (i = 0; i < 200; i++)
{
    for (j = 0; j < 200; j++)
    {
        elem_c = 0;
        for (r = 0; r < 200; r++)
        {
            elem_c = elem_c + a[i][r] * b[r][j];
            c[i][j] = elem_c;
        }
    }
}

```

Сложность $O(n^3)$.

- Для оценки времени выполнения программы модифицировал исходный код программы добавив подсчет времени а так же еще один цикл для последовательной проверки, а так же вывод количества секунд.

```

int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000 };
int count = sizeof(sizes) / sizeof(sizes[0]);

srand((unsigned)time(NULL));

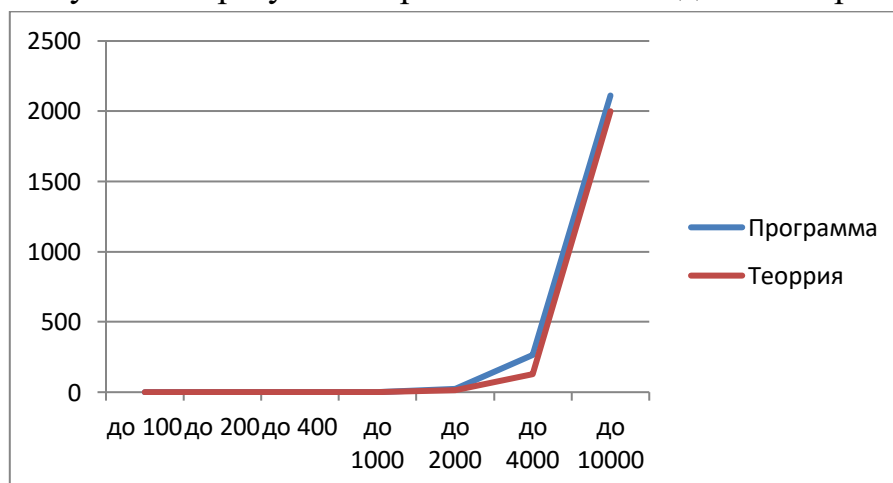
for (int s = 0; s < count; s++)

end = clock();

printf("Matric: %d x %d\n", n, n);
printf("Time: %.6f sec\n\n", (double)(end - start) / CLOCKS_PER_SEC);

```

- Полученный результат практически совпадает с теорией.



Задание 2:

Тип массива	Shell	Qs	Qsort
Случайный	0.002 сек	0.001 сек	0.003 сек
Возрастающий	0.00 сек	0.001 сек	0.002 сек
Убывающий	0.005 сек	0.00 сек	0.002 сек
Возр. и Убыв.	0.002 сек	0.01 сек	0.006 сек

Для выполнения второго задания потребовалось добавить переменную temp для записи в него массива не меняя исходный. А так же 4 цикла для записи в массив items.

```
srand((unsigned)time(NULL));
for (int i = 0; i < count; i++) {
    items[i] = rand() % 10000;
}
for (int i = 0; i < count; i++){
    temp[i] = items[i];
}
```

Результат работы программы

```
Random massiv
Shell_Time 0.002000 sec
Qs_Time 0.001000 sec
Qsort_Time 0.003000 sec
Upper Massiv
Shell_Time 0.000000 sec
Qs_Time 0.001000 sec
Qsort_Time 0.002000 sec
Down Massiv
Shell_Time 0.005000 sec
Qs_Time 0.000000 sec
Qsort_Time 0.002000 sec
UPandDownm Massiv
Shell_Time 0.002000 sec
Qs_Time 0.010000 sec
Qsort_Time 0.006000 sec
```

Вывод

В ходе лабораторной работы было измерено время выполнения алгоритмов умножения матриц и сортировки массивов. Установлено, что умножение матриц имеет сложность $O(n^3)$, а время сортировки зависит от используемого алгоритма и типа исходного массива. Полученные результаты позволяют сравнить практическое время работы алгоритмов с их теоретической сложностью.